5. Construct a Scheduling Program with C that Selects the Waiting Process with the Highest Priority to Execute Next (Priority Scheduling)

Code - Part 1 (Lines 1–34)

```
1  #include <stdio.h>
2
3  struct priority_scheduling {
4      char process_name;
5      int burst_time;
6      int waiting_time;
7      int turn_around_time;
8      int priority;
9  };
10
11 int main()
12 {
13     int number_of_process;
14     int total = 0;
15     int position;
16     float average_waiting_time, average_turnaround_time;
17
18     printf("Enter the total number of Processes: ");
19     scanf("%d", &number_of_process);
20
21     struct priority_scheduling process[number_of_process];
22     struct priority_scheduling temp_process;
23
24     int ASCII_number = 65;
25
26     printf("\nEnter Burst Time and Priority for each process:\n");
27
28     for (int i = 0; i < number_of_process; i++)
29     {
30         process[i].process_name = (char)ASCII_number;
31
32         printf("\nProcess %c\n", process[i].process_name);
33
34         printf("Burst Time: ");
```

Code - Part 2 (Lines 35–68)

```
35          scanf("%d", &process[i].burst_time);
36
37          printf("Priority: ");
38          scanf("%d", &process[i].priority);
39
40          ASCII_number++;
41      }
42
43      // Sort by priority (higher number = higher priority)
44      for (int i = 0; i < number_of_process; i++)
45      {
46          position = i;
47          for (int j = i + 1; j < number_of_process; j++)
48          {
49              if (process[j].priority > process[position].priority)
50                  position = j;
51          }
52
53          temp_process = process[i];
54          process[i] = process[position];
55          process[position] = temp_process;
56      }
57
58      // Waiting time
59      process[0].waiting_time = 0;
60      for (int i = 1; i < number_of_process; i++)
61      {
62          process[i].waiting_time = 0;
63
64          for (int j = 0; j < i; j++)
65              process[i].waiting_time += process[j].burst_time;
66
67          total += process[i].waiting_time;
68      }
```

Code - Part 3 (Lines 69–97)

```
69
70      average_waiting_time = (float)total / number_of_process;
71
72      // Turnaround time
73      total = 0;
74
75      printf("\nProcess\tBT\tWT\tTAT\n");
76
77      for (int i = 0; i < number_of_process; i++)
78      {
79          process[i].turn_around_time =
80              process[i].burst_time + process[i].waiting_time;
81
82          total += process[i].turn_around_time;
83
84          printf("%c\t%d\t%d\t%d\n",
85                 process[i].process_name,
86                 process[i].burst_time,
87                 process[i].waiting_time,
88                 process[i].turn_around_time);
89      }
90
91      average_turnaround_time = (float)total / number_of_process;
92
93      printf("\nAverage Waiting Time = %.2f", average_waiting_time);
94      printf("\nAverage Turnaround Time = %.2f\n", average_turnaround_time);
95
96      return 0;
97  }
```

Output

```
Enter the total number of Processes: 3

Enter Burst Time and Priority for each process:

Process A
Burst Time: 5 2
Priority:
Process B
Burst Time: 3 1
Priority:
Process C
Burst Time: 1 3
Priority:
Process BT      WT      TAT
C       1       0       1
A       5       1       6
B       3       6       9

Average Waiting Time = 2.33
Average Turnaround Time = 5.33


...Program finished with exit code 0
Press ENTER to exit console.
```